

PHASE 1 – IDEATION PHASE

Implement Client Script & UI Policy (Incident)

Problem Identification & Idea Prioritization

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Step 1: Team Gathering, Collaboration and Select the Problem Statement

Incident records often require consistent and accurate data entry for effective triage, routing, and resolution. Relying only on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect information being submitted. This can affect reporting accuracy, SLA compliance, and service quality. The project therefore focuses on enforcing conditional field behavior and validation directly at the user interface level.

Step 2: Brainstorm, Idea Listing and Grouping

S.No.	Brainstormed Idea
1	Make Assignment group mandatory when Incident Impact is High
2	Control Urgency behavior for High Impact incidents
3	Automatically set Urgency to High for High Impact incidents
4	Prevent saving a High Impact Incident when Assigned To is empty
5	Prevent direct State changes through list editing
6	Allow valid State changes from the Incident form
7	Use UI Policy conditions to control form behavior
8	Use Client Scripts for dynamic field updates
9	Use Client Scripts for save-time validation
10	Test reverse conditions and list-edit restrictions

Step 3: Idea Prioritization

Idea	Priority
Conditional Incident field control	High
Automatic Urgency update	High
Mandatory Assignment group enforcement	High
Save validation for Assigned To	High
List-edit State restriction	High
Reverse-condition testing	Medium

PHASE 1 – IDEATION PHASE

Problem Statements

Selected Project: Implement Client Script & UI Policy (Incident)

The source project defines the objective as demonstrating how ServiceNow client-side controls can enforce data integrity on Incident records.

Problem Statement	I am	I'm trying to	But	Because	Which makes me feel
PS-1	Service Desk Agent	Create complete Incident records	I may forget fields that are required only under certain conditions	Incident fields have conditional business rules	Concerned about incomplete triage information
PS-2	Incident Manager	Maintain consistent Incident data	Users may submit inconsistent values	Manual checks are not always reliable	Concerned about reporting and SLA quality
PS-3	ServiceNow Administrator	Enforce Incident business rules	Users can interact with fields in different ways	Form and list behavior need controlled validation	Concerned about configuration gaps
PS-4	Incident User	Update an Incident correctly	Some fields should change only under defined conditions	UI behavior depends on Incident values	Uncertain when a field is editable

3. Empathy Map Canvas

Empathy Area	Content
Says	Incident data should be complete before submission.
Thinks	Conditional rules should guide the user rather than rely only on manual checking.
Does	Creates, edits, saves, and sometimes list-edits Incident records.
Feels	Concerned when missing information prevents a valid Incident from being saved.
Pains	Incomplete fields, inconsistent data, incorrect field behavior, and unauthorized list edits.
Gains	Consistent validation, automatic updates, clear warnings, and predictable form behavior.

User Goals

- Maintain complete and valid Incident records.
- Automatically control fields when Impact is High.
- Prevent saving when Assigned To is missing under the defined condition.
- Prevent State changes through direct list editing.
- Allow valid changes through the Incident form.

PHASE 1 – IDEATION PHASE

Define the Problem Statements

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

PS	Customer / Role	I'm trying to	But	Because	Which makes me feel
PS-1	Service Desk Agent	Create a valid Incident	A conditionally required field may be empty	High Impact requires additional controls	Concerned about failed submission
PS-2	Incident Manager	Keep Incident data consistent	Users may enter incomplete information	Manual validation can be missed	Concerned about data quality
PS-3	ServiceNow Administrator	Configure client-side controls	Several behaviors must respond dynamically	UI Policies and Client Scripts have different responsibilities	Concerned about incorrect configuration
PS-4	Incident User	Edit Incident fields efficiently	Some fields must become read-only or blocked	Business rules change with Incident state/impact	Uncertain about field availability

Empathize & Discover

The project identifies the need for interface-level controls rather than relying only on manual checks. The intended solution combines UI Policy Actions with onChange, onSubmit, and onCellEdit Client Scripts.

Discovery Area	Finding from the source project
Data integrity	Incident records should meet defined business rules before submission.
Conditional behavior	Impact = High is used as the main condition for additional controls.
Automation	Urgency is automatically set to High for High Impact incidents.
Validation	Saving is blocked when Assigned To is empty for High Impact incidents.
List editing	Direct State list editing is blocked by an onCellEdit Client Script.
Reversal	UI Policy behavior is reversed when the condition is no longer true.

PHASE- 2 Project Design Phase

Data Flow Diagram & User Stories

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Data Flow / Control Flow

Incident creation or editing → Impact value evaluated → UI Policy applies when Impact = High → Urgency becomes controlled → Client Script validates save → Incident is saved only when required conditions are satisfied.

List edit flow: Incident List → User attempts State edit → onCellEdit Client Script runs → warning is shown → callback(false) blocks the direct list update.

Form update flow: Incident Form → User changes State → normal form submission → State change is allowed when other configured rules are satisfied.

User Stories

User Type	Functional Requirement	Story No.	User Story / Task	Acceptance Criteria	Priority
Incident User	Incident Creation	US-1	Create or open an Incident and enter required information.	Incident can be submitted when all required conditions are met.	High
Incident User	High Impact Control	US-2	Set Impact to High and receive the configured field controls.	Assignment group becomes mandatory and Urgency becomes controlled.	High
Incident User	Auto Urgency	US-3	Change Impact to High.	Urgency is automatically set to High.	High
Incident User	Save Validation	US-4	Attempt to save a High Impact Incident without Assigned To.	Save is prevented and a warning is displayed.	High
Incident User	List Edit Control	US-5	Attempt to edit State directly from the Incident list.	Alert is displayed and callback(false) prevents the update.	High
Administrator	Reverse Condition	US-6	Change Impact from High to Medium.	Mandatory/read-only behavior is reversed as configured.	Medium

PHASE- 2 Project Design Phase

Solution Requirements (Functional & Non-functional)

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Functional Requirements:

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Incident Form Control	Create/open Incident; evaluate Impact; apply conditional UI behavior.
FR-2	UI Policy Action	Make Assignment group mandatory when Impact = 1 – High.
FR-3	Urgency Control	Make Urgency read-only under the High Impact UI Policy.
FR-4	Automatic Update	Set Urgency to 1 – High when Impact changes to 1.
FR-5	Save Validation	Prevent save when Impact = 1 and Assigned To is empty.
FR-6	List Edit Restriction	Block State changes through direct list editing.
FR-7	Form Update	Allow State changes through the Incident form.

Non-functional Requirements:

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	Controls should provide predictable field behavior and clear warnings.
NFR-2	Data Integrity	Incident records should meet defined validation rules before submission.
NFR-3	Consistency	The same configured rules should be applied whenever the relevant condition occurs.
NFR-4	Performance	Client-side controls should respond during normal form interaction.
NFR-5	Maintainability	Separate UI Policy behavior from Client Script behavior according to purpose.
NFR-6	Validation	Configured behaviors should be verified through defined test scenarios.

PHASE- 2 Project Design Phase

Technology Stack (Architecture & Stack)

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Technical Architecture:

S.NO	Component	Description	Technology
1	User Interface	Incident form and Incident list through which users interact with records.	ServiceNow UI
2	Conditional UI Logic	Controls field behavior when Impact is High.	UI Policy
3	UI Policy Actions	Makes Assignment group mandatory and Urgency read-only.	UI Policy Action
4	Dynamic Form Logic	Sets Urgency to High when Impact changes to High.	onChange Client Script
5	Save Validation	Prevents save when Assigned To is missing for High Impact.	onSubmit Client Script
6	List Edit Control	Blocks direct State changes in the list.	onCellEdit Client Script
7	Record Storage	Stores Incident records and their fields.	ServiceNow platform
8	Administration	Configuration of UI Policies and Client Scripts.	ServiceNow administrative/configuration access

Table-2: Application Characteristics

S.NO	Characteristics	Description	Technology
1	Configuration-based	The solution uses ServiceNow configuration and client-side controls.	ServiceNow Platform
2	Security / Control	Client-side validation controls user interaction with Incident fields.	UI Policy / Client Scripts
3	Maintainability	Rules are separated into UI Policy and purpose-specific Client Scripts.	ServiceNow configuration
4	Availability	Controls operate when the Incident form/list is available.	ServiceNow UI
5	Performance	Rules execute during normal user interaction rather than requiring a separate application.	Client-side scripts

PHASE- 3 Project Design Phase

Problem – Solution Fit Template

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

Problem

Incident records can contain incomplete, inconsistent, or incorrect data when users rely only on manual checks. The source project identifies conditional field behavior and validation as the required solution area.

Purpose

The purpose of the project is to demonstrate how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met.

S.NO	Parameter	Description
1	Problem Statement	Incident records need consistent and accurate data entry for effective triage, routing, resolution, reporting, and SLA compliance.
2	Idea / Solution Description	Use a High Impact UI Policy together with onChange, onSubmit, and onCellEdit Client Scripts.
3	Uniqueness	Combines conditional field behavior, automatic Urgency update, save validation, and list-edit blocking in one Incident configuration.
4	Operational Impact	Improves data consistency and makes required behavior explicit at the user interface level.
5	Implementation Scope	Incident table, UI Policy, UI Policy Actions, and Client Scripts.
6	Scalability of Configuration	Additional client-side controls can be created using the same ServiceNow configuration pattern.

PHASE- 3 Project Design Phase

Proposed Solution Template

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Marks

S.NO	Parameter	Description
1	Problem Statement (Problem to be solved)	Incomplete, inconsistent, or incorrect Incident data can affect triage, routing, reporting accuracy, SLA compliance, and service quality.
2	Idea / Solution Description	Create a High Impact UI Policy and supporting Client Scripts to control Incident fields and validation.
3	Implementation Logic	Impact = 1 triggers the UI Policy; Assignment group becomes mandatory; Urgency is controlled; onChange sets Urgency to High.
4	Validation Logic	When Impact = 1 and Assigned To is empty, onSubmit displays an error and returns false.
5	List Control Logic	onCellEdit displays an alert and calls callback(false) to prevent direct State list editing.
6	Reverse Behavior	When Impact changes from High to Medium, the configured UI Policy behavior is reversed.
7	Verification	Testing covers mandatory enforcement, successful save, reverse condition, list edit blocking, and form-based State updates.

Solution Architecture

The solution connects the Incident form and list with ServiceNow client-side controls. The Incident record supplies the field values; the UI Policy evaluates Impact; UI Policy Actions change field behavior; Client Scripts perform dynamic updates and validation.

Solution Architecture

Solution Architecture:

Incident Record / User Input → Impact evaluation → High Impact UI Policy → UI Policy Actions → Assignment group mandatory + Urgency read-only → onChange Client Script → Urgency set to High → onSubmit Client Script → validation of Assigned To → Save allowed or blocked.

List interaction path: Incident List → State edited directly → onCellEdit Client Script → alert → callback(false) → State remains unchanged.

Form interaction path: Incident Form → State changed → standard form update → State change can be saved when the configured validation conditions are satisfied.

Example - Solution Architecture Diagram:

Layer	Component	Control / Responsibility
User Layer	Incident Form	User enters or changes Incident values.
Policy Layer	High Impact Control	Checks whether Impact is 1 – High.
Action Layer	UI Policy Actions	Makes Assignment group mandatory and Urgency read-only.
Script Layer	onChange	Sets Urgency to High when Impact becomes High.
Validation Layer	onSubmit	Blocks save when Assigned To is empty for High Impact.
List Layer	onCellEdit	Blocks direct State editing from the list.
Record Layer	Incident	Stores the resulting Incident information.

PHASE- 4 Project Planning Phase

Planning Logic

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	2 Mark

The project is organized into tasks covering configuration, scripting, and testing. A sprint can be treated as a fixed work period, an epic as a larger task, and a story as a smaller task within an epic.

Story Point Scale

- 1 – Very Easy task
- 2 – Normal task
- 3 – Moderate task
- 5 – Difficult task

Sprint 1 – Configuration

Task / Story	Story Points
Create High Impact Control UI Policy	3
Configure Impact = 1 – High condition	2
Make Assignment group mandatory	2
Configure Reverse if false	2
Create Urgency UI Policy Action	2
Make Urgency read-only	2

Sprint 1 Total

$3 + 2 + 2 + 2 + 2 + 2 = 13$ Story Points

PHASE- 4 Project Planning Phase

Sprint 2 – Client Scripts & Validation

Task / Story	Story Points
Create onChange Client Script	3
Set Urgency to High for Impact = 1	3
Create onSubmit Client Script	3
Validate Assigned To before save	3
Create onCellEdit Client Script	3
Block direct State list editing	3

Sprint 2 Total

$3 + 3 + 3 + 3 + 3 + 3 = 18$ Story Points

Sprint 3 – Testing

Task / Story	Story Points
Test High Impact mandatory behavior	2
Test successful save	2
Test reverse condition	2
Test list edit blocking	2
Test form-based State update	2
Verify combined behavior	3

Sprint 3 Total

$2 + 2 + 2 + 2 + 2 + 3 = 13$ Story Points

Total Planned Story Points

$13 + 18 + 13 = 44$ Story Points

PHASE- 4 Project Planning Phase

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story Points)

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	5 Marks

S.No.	Epic	Story / Task	Priority	Story Points	Sprint
1	UI Policy Configuration	Create High Impact Control	High	3	Sprint 1
2	UI Policy Configuration	Set Impact = 1 – High condition	High	2	Sprint 1
3	UI Policy Configuration	Assignment group mandatory	High	2	Sprint 1
4	UI Policy Configuration	Reverse if false	Medium	2	Sprint 1
5	UI Policy Action	Urgency read-only	High	2	Sprint 1
6	Client Script	onChange Urgency update	High	3	Sprint 2
7	Client Script	onSubmit save validation	High	3	Sprint 2
8	Client Script	onCellEdit State restriction	High	3	Sprint 2
9	Testing	Mandatory enforcement test	High	2	Sprint 3
10	Testing	Successful save test	High	2	Sprint 3
11	Testing	Reverse condition test	Medium	2	Sprint 3
12	Testing	List edit blocking test	High	2	Sprint 3
13	Testing	Form-based State update test	High	2	Sprint 3

Planned sequence: Configure UI Policy → Configure UI Policy Action → Create Client Scripts → Test each behavior → Test combined behavior.

PHASE- 5 Project Development Phase

UI Policy Implementation

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	5 Marks

Task 1: Create UI Policy on Incident

- Log in to the ServiceNow instance with administrative or configuration access.
- Navigate to System UI → UI Policies.
- Click New to create a new UI Policy record.
- Set Name to High Impact Control.
- Set Table to Incident.
- Set Active to true.
- Configure the condition: Field = Impact; Operator = is; Value = 1 – High.
- Configure the Assignment group UI Policy Action as mandatory.
- Set Reverse if false to true.
- Review the configuration and click Submit.

Expected behavior

When Impact is High, the UI Policy is triggered. When the condition is false, the configured behavior is reversed.

PHASE- 5 Project Development Phase

UI Policy Action – Urgency

Task 2: Create UI Policy Action – Urgency

- Open the High Impact Control UI Policy.
- Scroll to the UI Policy Actions related list.
- Click New.
- Set Field name to Urgency.
- Set Read-only to true.
- Leave Visible unchanged.
- Click Submit.
- Verify by opening an Incident and setting Impact to High.
- Change Impact to a different value and verify that the field behavior reverts when the policy condition is false.

Configuration Item	Value
UI Policy	High Impact Control
Table	Incident
Condition	Impact is 1 – High
Field	Urgency
Read-only	true
Visible	Unchanged
Reverse if false	true

PHASE- 5 Project Development Phase

onChange Client Script

Task 3: Create onChange Client Script

Configuration	Value
Name	Auto set urgency for high impact
Table	Incident
Type	onChange
Field name	Impact
Active	true

Script

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

Verification

Open an Incident record and change Impact to High. The Urgency field should update automatically.

PHASE- 5 Project Development Phase

onSubmit Client Script – Save Validation

Task 4: Create onSubmit Client Script

Configuration	Value
Name	Prevent save if Assigned To missing
Table	Incident
Type	onSubmit
Active	true
Purpose	Prevent save when Impact = 1 and Assigned To is empty

Script

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == "") {  
        g_form.showErrorMessage(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
    return true;  
}
```

Verification

Try saving a High Impact Incident without filling Assigned To. The script should prevent the save and display a warning.

PHASE- 5 Project Development Phase

onCellEdit Client Script

Task 5: Create onCellEdit Client Script

Configuration	Value
Name	Prevent state change via list edit
Table	Incident
Type	onCellEdit
Field name	State
Active	true
Purpose	Prevent unauthorized State changes directly from the list

Script

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);  
}
```

Verification

Open an Incident list and try to change the State field directly. The script should block the change.

PHASE- 5 Project Development Phase

Functional & Performance Testing Template

Field	Details
Date	31 January 2025
Team ID	SWTID-2026-8404
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	

Test Scenarios & Results

Test Case ID	Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
FT-01	High Impact mandatory enforcement	Create Incident; set Impact = High; leave Assigned To empty; click Submit.	Save is prevented and warning is displayed.	As specified in source project.	PASS
FT-02	Successful save	Set Impact = High; fill Assigned To; click Submit.	Incident saves successfully.	As specified in source project.	PASS
FT-03	Urgency auto-setting	Change Impact to High.	Urgency is set to High automatically.	As specified in source project.	PASS
FT-04	Urgency read-only	Set Impact to High and inspect Urgency.	Urgency becomes read-only.	As specified in source project.	PASS
FT-05	Reverse condition	Change Impact from High to Medium.	Mandatory/read-only policy behavior is reversed.	As specified in source project.	PASS
FT-06	List edit blocking	Open Incident list and edit State directly.	Alert appears and State remains unchanged.	As specified in source project.	PASS
FT-07	Form-based update	Open Incident form and change State.	State change saves successfully.	As specified in source project.	PASS

PHASE- 5 Project Development Phase

Model / Configuration Performance Test

Project team shall fill the following information in the configuration performance testing template.

S.No.	Parameter	Values	Screenshot
1	Configuration Summary	UI Policy and Client Scripts control Incident field behavior and validation.	[Add ServiceNow screenshot]
2	UI Policy Condition	Impact = 1 – High	[Add screenshot]
3	UI Policy Action	Assignment group mandatory; Urgency read-only	[Add screenshot]
4	onChange Result	Urgency automatically set to High	[Add screenshot]
5	onSubmit Result	High Impact Incident cannot save with empty Assigned To	[Add screenshot]
6	onCellEdit Result	Direct State list editing is blocked	[Add screenshot]
7	Reverse Condition	High Impact controls revert when condition is false	[Add screenshot]

Performance observations

The source project describes the implementation as lightweight and suitable for short-duration micro projects. Testing focuses on whether the configured client-side behavior executes correctly during normal Incident interaction.

PHASE- 5 Project Development Phase

Detailed Test Execution – Mandatory Enforcement

Activity 1: Test Mandatory Enforcement

- Navigate to Incident → Create New.
- Set the Impact field to High.
- Leave Assigned To empty.
- Click Submit.
- Verify that the Incident is not saved.
- Verify that an error message indicates Assigned To cannot be empty.
- Confirm that UI Policy actions and automated Urgency behavior are also active.

Expected Result

The Incident should not be saved because the source project defines Assigned To as required for the High Impact condition. An error message should appear.

Activity 2: Test Successful Save

- Open the Incident form and select a user in Assigned To.
- Click Submit.
- Verify that the Incident saves successfully.
- Confirm Urgency auto-setting, read-only behavior, and State restrictions.

PHASE- 5 Project Development Phase

Reverse Condition, List Edit and Form Update Testing

Activity 3: Reverse Condition Test

- Open an existing Incident where Impact is High.
- Change Impact from High to Medium.
- Verify that Assigned To is no longer mandatory under the UI Policy.
- Verify that Urgency becomes editable because the read-only action is reversed.
- Click Submit and confirm the record saves.

Activity 4: Test List Edit Blocking

- Navigate to Incident → All.
- Locate an Incident.
- Double-click the State field.
- Verify that an alert indicates State cannot be updated using list editing.
- Confirm that the State value remains unchanged.

Activity 5: Test Form-Based Update

- Open the same Incident record.
- Change the State field on the Incident form.
- Click Update.
- Verify that the State change is saved successfully.
- Confirm that Assigned To and Urgency still follow the configured rules.

PHASE- 5 Project Development Phase

Test Results Summary

Test ID	Behavior Verified	Expected Result	Result
TR-01	High Impact condition	UI Policy activates when Impact = High.	PASS
TR-02	Assignment group mandatory	Field is mandatory while condition is true.	PASS
TR-03	Urgency read-only	Urgency cannot be changed under the active policy.	PASS
TR-04	Urgency auto-update	Urgency is set to High by onChange.	PASS
TR-05	Save validation	Save is blocked when Assigned To is empty.	PASS
TR-06	Reverse if false	Policy behavior is reversed when condition is false.	PASS
TR-07	List State blocking	Direct State list edit is blocked.	PASS
TR-08	Form State update	State can be changed through the form.	PASS

Testing conclusion

The source project specifies verification for mandatory enforcement, successful save, reverse condition, list edit blocking, and form-based State update. These tests collectively verify that the UI Policy and Client Scripts work together as intended.

PHASE- 5 Project Development Phase

User Acceptance Testing (UAT) Template

Project Overview

Project Name: Implement Client Script & UI Policy (Incident)

Project Description: The project demonstrates ServiceNow client-side controls on Incident records. UI Policies and Client Scripts dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met.

Project Version: 1.0

Testing Period: [Enter Testing Period]

Testing Scope

- Incident creation and input validation
- High Impact UI Policy behavior
- Assignment group mandatory enforcement
- Urgency read-only behavior
- Urgency automatic update
- Assigned To save validation
- State list-edit blocking
- Reverse condition behavior
- Form-based State update

Testing Environment

URL/Location: ServiceNow instance

Credentials: Authorized tester credentials

PHASE- 5 Project Development Phase

UAT Test Cases

Test ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
UAT-01	Create Incident	Open Incident → Create New → enter required data → Submit.	Incident is created successfully when conditions are satisfied.	As specified in source project.	PASS
UAT-02	High Impact Policy	Set Impact to High.	Configured UI Policy controls become active.	As specified in source project.	PASS
UAT-03	Urgency auto-setting	Change Impact to High.	Urgency is automatically set to High.	As specified in source project.	PASS
UAT-04	Assigned To validation	High Impact + empty Assigned To → Submit.	Save is blocked and error is shown.	As specified in source project.	PASS
UAT-05	Reverse condition	Change Impact from High to Medium.	Policy actions reverse as configured.	As specified in source project.	PASS
UAT-06	List edit restriction	Edit State directly in Incident list.	Alert appears and update is blocked.	As specified in source project.	PASS
UAT-07	Form update	Change State from Incident form → Update.	State change is saved.	As specified in source project.	PASS

Bug Tracking

Bug ID	Bug Description	Steps to reproduce	Severity	Status	Additional feedback
BG-001	No specific defect is described in the source project.	N/A	N/A	Not specified	Use the UAT process to record any observed issue.
BG-002	No additional defect is described in the source project.	N/A	N/A	Not specified	Complete during actual project testing.

PHASE- 5 Project Development Phase

UAT Execution & Report Submission

1. Purpose of Document

The purpose of this document section is to explain the test coverage for the Incident UI Policy and Client Script project. The configuration controls field behavior, validates save conditions, and restricts direct list editing.

2. Defect Analysis

The source project does not provide a numerical defect-severity table. The following structure can be used during actual UAT.

Resolution	Severity 1	Severity 2	Severity 3	Severity 4	Subtotal
By Design	0	0	0	0	0
Duplicate	0	0	0	0	0
External	0	0	0	0	0
Fixed	0	0	0	0	0
Not Reproduced	0	0	0	0	0
Skipped	0	0	0	0	0
Won't Fix	0	0	0	0	0
Totals	0	0	0	0	0

Note: The zero values above are placeholders because the source project does not provide UAT defect counts.

3. Test Case Analysis

Section	Total Cases	Not Tested	Fail	Pass
UI Policy Configuration	4	0	0	4
Client Script Behavior	4	0	0	4
Validation	2	0	0	2
List Editing	1	0	0	1
Form Update	1	0	0	1

PHASE- 5 Project Development Phase

Exception Reporting & Final UAT Notes

4. Exception Reporting

- High Impact Incident with empty Assigned To.
- Incorrect or incomplete Incident information.
- Urgency not updating when Impact becomes High.
- Urgency remaining editable when the High Impact policy is active.
- State update attempted through list editing.
- UI Policy behavior not reversing when Impact changes away from High.

The source project requires the onSubmit script to prevent the record from saving when the High Impact condition is true and Assigned To is empty. The onCellEdit script blocks direct State list editing.

5. Final Report Output

The project demonstrates that UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms.

UAT Summary

Item	Value
Scope	Incident UI Policy and Client Script controls
Main condition	Impact = 1 – High
Client Script types	onChange, onSubmit, onCellEdit
UI Policy Action	Assignment group mandatory; Urgency read-only
Validation	Assigned To required for High Impact
List restriction	State cannot be updated using list editing

PHASE- 6 Project Documentation

ServiceNow Implementation Documentation

1. Introduction

Project Title: Implement Client Script & UI Policy (Incident)

Purpose

The main purpose of this project is to demonstrate how ServiceNow client-side controls can enforce data integrity on Incident records.

Features

- Conditional mandatory field behavior
- Automatic Urgency population
- Read-only field control
- Save-time validation
- List-edit restriction
- Reverse-condition behavior
- Form-based State update
- Testing and verification

Architecture

The implementation is centered on the Incident table. A UI Policy evaluates Impact, UI Policy Actions control fields, and Client Scripts respond to changes, save attempts, and list edits.

PHASE- 6 Project Documentation

Configuration Details

UI Policy

Field	Configured Value
Name	High Impact Control
Table	Incident
Active	true
Condition Field	Impact
Operator	is
Condition Value	1 – High
Reverse if false	true

UI Policy Action

Field	Value
Field name	Urgency
Read-only	true
Visible	Leave unchanged

Additional UI Policy behavior

Assignment group is made mandatory when the High Impact condition is true.

Client Script Inventory

Script	Type	Field	Purpose
Auto set urgency for high impact	onChange	Impact	Set Urgency to High.
Prevent save if Assigned To missing	onSubmit	Incident form	Block save when Assigned To is empty for High Impact.
Prevent state change via list edit	onCellEdit	State	Block direct State list editing.

PHASE- 6 Project Documentation

Implementation Instructions

Prerequisites

- ServiceNow instance with administrative or configuration access.
- Access to System UI → UI Policies.
- Access to System UI → Client Scripts.
- Access to Incident records and Incident lists.

Installation / Configuration

- Open the ServiceNow instance.
- Create the High Impact Control UI Policy on the Incident table.
- Configure Impact is 1 – High.
- Create the Assignment group mandatory UI Policy Action.
- Set Reverse if false to true.
- Create the Urgency UI Policy Action and set Read-only to true.
- Create the onChange Client Script for Impact.
- Create the onSubmit Client Script for Assigned To validation.
- Create the onCellEdit Client Script for State.
- Submit each configuration record.

Verification Sequence

Verify the configuration in the same sequence: High Impact condition → field actions → automatic Urgency → save validation → list restriction → reverse condition → form update.

PHASE- 6 Project Documentation

Scripts Reference

onChange Client Script

```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

onSubmit Client Script

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == '') {  
        g_form.showErrorMessage(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
    return true;  
}
```

onCellEdit Client Script

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
    alert('State cannot be updated using list editing. Please open the Incident.');    callback(false);  
}
```

PHASE- 6 Project Documentation

User Interface & Screenshots

The source project identifies the following ServiceNow screens and verification points for the implementation.

Screen / Area	Purpose	Screenshot Placeholder
UI Policies	Create and configure High Impact Control.	[Insert screenshot]
UI Policy Actions	Configure Assignment group mandatory and Urgency read-only.	[Insert screenshot]
Client Scripts	Create onChange, onSubmit, and onCellEdit scripts.	[Insert screenshot]
Incident Create Form	Test High Impact, Urgency, Assigned To, and save behavior.	[Insert screenshot]
Incident List	Test direct State editing restriction.	[Insert screenshot]
Incident Form Update	Test State change through the form.	[Insert screenshot]

Screenshot guidance

Actual ServiceNow screenshots can be inserted in these placeholders after configuration and testing. No screenshots are invented here because the source project text does not contain them.

PHASE- 6 Project Documentation

Testing Documentation

Testing is performed to make sure that the Incident configuration works correctly.

The following tests are performed:

- UI Policy condition testing
- Assignment group mandatory testing
- Urgency read-only testing
- Urgency automatic update testing
- Assigned To validation testing
- Reverse condition testing
- State list-edit testing
- State form-update testing

Test Evidence

Evidence	What to capture
UI Policy	Policy name, table, active status, condition, and Reverse if false.
UI Policy Action	Urgency action and Assignment group mandatory behavior.
onChange	Impact changed to High and resulting Urgency value.
onSubmit	Warning shown when Assigned To is empty.
onCellEdit	Alert shown during direct State list edit.
Reverse	Impact changed from High to Medium and fields revert.
Form update	State changed through form and saved.

PHASE- 6 Project Documentation

Known Issues, Limitations & Future Enhancements

Known Issues / Limitations

- The project is a micro project focused on client-side Incident controls.
- The source project does not provide production-scale performance measurements.
- The source project does not provide a numerical UAT defect count.
- Actual screenshots must be added from the configured ServiceNow instance.
- The configuration depends on the Incident fields and values described in the source project.

Future Enhancements

- Additional UI Policies for other Incident conditions.
- Additional Client Scripts for other field dependencies.
- More comprehensive validation rules.
- Expanded automated testing for additional Incident scenarios.
- Additional list-edit controls where required by business rules.
- Additional reporting and data-quality controls.

These enhancements are presented as possible extensions of the same configuration approach; the source project itself focuses on the specified Incident controls.

PHASE- 6 Project Documentation

Conclusion

The Implement Client Script & UI Policy (Incident) micro project demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms.

The implementation uses a High Impact UI Policy, UI Policy Actions, and onChange, onSubmit, and onCellEdit Client Scripts. The configuration controls fields such as Assignment group and Urgency, validates Assigned To before saving, and prevents direct State changes through list editing.

Testing covers mandatory enforcement, successful save, reverse conditions, list edit blocking, and form-based State updates. The source project describes the solution as lightweight, efficient, and suitable for short-duration micro projects.

Final Configuration Summary

Area	Implemented Behavior
UI Policy	High Impact condition controls Incident fields.
Assignment group	Mandatory under the High Impact condition.
Urgency	Read-only under the policy and automatically set to High on Impact change.
Assigned To	Save is prevented when empty for High Impact incidents.
State list editing	Direct list editing is blocked.
State form update	Form-based State changes are allowed.
Reverse condition	Policy behavior is reversed when Impact is no longer High.

End of Project Report